

Jogo 21 com Java RMI

Sistemas Distribuídos / Programação Paralela – UTFPR

Bruno Nunes da Silva

Brunosilva.2022@alunos.utfpr.edu.br

Entony Cesar Siminhuk

entony@alunos.utfpr.edu.br

2 de julho de 2026

Resumo

Este trabalho apresenta a implementação de uma aplicação em Java que utiliza a Invocação de Método Remoto (RMI) para criar um jogo de cartas 21, operando em tempo real sob uma arquitetura Cliente/Servidor. O sistema conta com validação de identidade, isolamento completo de partidas e uma interface gráfica interativa para os usuários.

Palavras-chave: RMI, Jogo 21, Threads, Java Swing, Interface Gráfica.

1 Introdução

- O presente trabalho simula o jogo de cartas 21 utilizando Java RMI. O sistema implementa um modelo cliente-servidor onde os jogadores realizam seus jogos individuais interagindo de forma distribuída com o servidor.

2 Descrição do Problema

O desafio consiste em projetar e implementar um ambiente de jogo confiável onde múltiplos clientes possam interagir individualmente com o mesmo servidor sem que o estado das cartas ou o saldo de um jogador interfira nas demais sessões concorrentes.

3 Funcionalidades

O sistema foi projetado para oferecer uma experiência completa do jogo Blackjack de forma distribuída. Suas principais funcionalidades englobam:

- **Controle de Acesso e Saldo Inicial:** Validação de unicidade para o nome do jogador no servidor e concessão automática de um saldo fictício inicial de R\$ 500 para apostas.
- **Mecanismo de Apostas:** Sistema que valida e deduz o valor apostado antes do início de cada rodada, recalculando o saldo ao final com base no resultado (vitória, derrota ou empate).
- **Lógica de Partida Isolada:** Inicialização de turnos individuais onde o jogador recebe suas cartas iniciais e interage através dos comandos clássicos de pedir carta (*Hit*) ou parar (*Stand*).

- **Inteligência da Banca (Dealer):** Automação das jogadas do *dealer* controlada estritamente pelo servidor, seguindo a regra de compra obrigatória até atingir 17 pontos ou mais.

3.1 Interface Gráfica (GUI)

Como diferencial de usabilidade e cumprimento dos requisitos opcionais do edital, foi desenvolvida uma interface gráfica baseada na biblioteca **Java Swing**.

A interface abstrai toda a complexidade da rede para o usuário, substituindo a interação via terminal por telas dinâmicas e intuitivas. O componente gráfico é responsável por:

- **Formulário de Entrada:** Tela inicial para conexão ao servidor RMI e validação em tempo real do apelido do jogador.
- **Renderização Visual das Cartas:** Exibição física das cartas do jogador e do *dealer* em um painel que simula uma mesa de cassino. O sistema gerencia dinamicamente os componentes visuais para ocultar a carta fechada da banca usando uma textura de verso de carta enquanto o jogo estiver em andamento.
- **Painel de Estado Síncrono:** Atualização imediata dos rótulos de pontuação, saldo atualizado, valor em risco na aposta e mensagens textuais de status decorrentes do processamento remoto (ex: “Você ganhou!”, “A banca estourou!”).
- **Controle por Botões:** Desativação e ativação contextual de botões de ação (*Hit/Stand*) para guiar o fluxo do usuário e impedir requisições inválidas fora do turno de jogo.

4 Implementação

4.1 Lógica

A lógica utilizada para o desenvolvimento do jogo multiusuário está alicerçada na infraestrutura do Java RMI e no uso de coleções seguras para concorrência (*Thread-Safe*).

Ao estender a classe `UnicastRemoteObject`, o construtor do servidor exporta o objeto remoto, abrindo portas de comunicação para ouvir requisições. Cada chamada efetuada por um cliente é executada em uma

thread separada alocada pelo *thread pool* interno do RMI.

Para evitar vazamentos de memória e inconsistências de estado, utilizou-se a estrutura `ConcurrentHashMap`. Ela garante que múltiplas linhas de execução paralelas travem atomicamente apenas o registro do jogador que está interagindo naquele milissegundo, sem interferir nas mesas adjacentes.

A segurança no encerramento das partidas foi estruturada de forma que o método de resolução execute a remoção da chave do usuário no mapa global de partidas ativas antes de serializar a resposta de fechamento. Isso libera recursos do servidor de forma imediata e previne requisições duplicadas. A persistência do saldo do usuário é mantida em um mapa separado, para ser usado ao término das rodadas.

4.2 Arquivos do Projeto

- **ServicoJogo21 (Interface Remota):** Define o contrato e a assinatura de todos os métodos que o cliente pode invocar remotamente. Estende a interface `Remote` e obriga todos os métodos a lançar `RemoteException`.
- **ServicoJogo21Impl (Servidor):** Implementa a interface remota e gerencia as regras do jogo. Contém as estruturas globais `partidasAtivas` e `saldosJogadores`. Centraliza a lógica de embaralhamento aleatório (via `Collections.shuffle()`) e o cálculo de pontuação flexível do Ás (computado inicialmente como 11 e convertido dinamicamente para 1 caso a mão estoure os 21 pontos).
- **Cliente Grafico / Cliente:** Responsáveis por localizar o serviço de rede através do RMI Registry na porta 1099. Enquanto o `Cliente` executa em modo texto no console, o `ClienteGrafico` implementa a árvore de componentes visuais do Swing para capturar eventos de clique e redesenhar a mesa com base no estado atualizado.
- **EstadoJogo (Modelo de Dados):** Representa o pacote de informações trafegado do Servidor para o Cliente. Implementa a interface `Serializable`, o que permite que suas instâncias sejam desestruturadas em fluxos de bytes, enviadas pela rede por meio de passagem por valor (cópia), e remontadas no cliente.
- **PartidaInterna (Classe Utilitária):** Estrutura de escopo privado que encapsula os dados voláteis de uma rodada específica, incluindo o baralho exclusivo de 52 cartas, a sublista do jogador, a sublista do dealer e a quantia que foi empenhada na mesa.

Referências

- [1] TANENBAUM, A. S. *Distributed Systems*.